Relative path: eval/metrics.py
Filename: metrics.py Extension: .py

```
"""
OASIS v2 — eval/metrics.py

Computes the numbers the report needs (REPORT_STRUCTURE.md Section 8):
- overall precision/recall/F1 on the imbalanced label set
- precision@top-1% (realistic analyst alert budget, per the brief)
- per-type classification accuracy / confusion matrix
- cold-start entity results
- concept-drift result: confirms `insider_drift` sessions are NOT
  permanently flagged once the profiler has adapted, with real before/after
  scores as evidence.
"""
from __future__ import annotations

from typing import Dict, List

import numpy as np


def precision_recall_f1(anomaly_events: List[dict], ground_truth: Dict[str, dict],
                        threshold: float = 0.5):
    tp = fp = fn = tn = 0
    for ev in anomaly_events:
        gt_label = ground_truth[ev["session_id"]]["label"]
        pred_attack = ev["anomaly_score"] >= threshold
        true_attack = gt_label == "attack"
        if pred_attack and true_attack:
            tp += 1
        elif pred_attack and not true_attack:
            fp += 1
        elif not pred_attack and true_attack:
            fn += 1
        else:
            tn += 1
    precision = tp / (tp + fp) if (tp + fp) else 0.0
    recall = tp / (tp + fn) if (tp + fn) else 0.0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) else 0.0
    return {"threshold": threshold, "tp": tp, "fp": fp, "fn": fn, "tn": tn,
            "precision": round(precision, 4), "recall": round(recall, 4), "f1": round(f1, 4)}


def precision_at_k_percent(anomaly_events: List[dict], ground_truth: Dict[str, dict],
                           k_percent: float = 1.0):
    """Precision among the top-k% highest-anomaly_score sessions — the
    brief's explicit 'realistic analyst alert budget'."""
    n = len(anomaly_events)
    k = max(1, round(n * k_percent / 100))
    ranked = sorted(anomaly_events, key=lambda e: -e["anomaly_score"])[:k]
    hits = sum(1 for ev in ranked if ground_truth[ev["session_id"]]["label"] == "attack")
    return {"k_percent": k_percent, "k_sessions": k,
            "precision_at_k": round(hits / k, 4), "hits": hits}


def classification_confusion_matrix(classified_events: List[dict], ground_truth: Dict[str, dict]):
    labels = sorted({ground_truth[c["session_id"]]["anomaly_type"] for c in classified_events}
                    | {c["anomaly_type"] for c in classified_events})
    idx = {l: i for i, l in enumerate(labels)}
    matrix = np.zeros((len(labels), len(labels)), dtype=int)
    for c in classified_events:
        true_type = ground_truth[c["session_id"]]["anomaly_type"]
        pred_type = c["anomaly_type"]
        matrix[idx[true_type], idx[pred_type]] += 1

    per_class = {}
    for l in labels:
        i = idx[l]
        tp = matrix[i, i]
        fp = matrix[:, i].sum() - tp
        fn = matrix[i, :].sum() - tp
        precision = tp / (tp + fp) if (tp + fp) else 0.0
        recall = tp / (tp + fn) if (tp + fn) else 0.0
        per_class[l] = {"precision": round(float(precision), 4),
                        "recall": round(float(recall), 4),
```

```
                        "support": int(matrix[i, :].sum())}
    return {"labels": labels, "matrix": matrix.tolist(), "per_class": per_class}


def cold_start_eval(anomaly_events: List[dict], sessions_by_id: Dict[str, dict]):
    """Buckets sessions by how many PRIOR sessions that entity had at
    scoring time (0 / 1-4 / 5+) and reports mean anomaly_score per bucket —
    the brief's explicit '0/1/5 prior sessions' cold-start check."""
    buckets = {"0_prior": [], "1_to_4_prior": [], "5_plus_prior": []}
    seen_count: Dict[str, int] = {}
    for ev in anomaly_events:
        eid = ev["entity_id"]
        n_prior = seen_count.get(eid, 0)
        if n_prior == 0:
            buckets["0_prior"].append(ev["anomaly_score"])
        elif n_prior < 5:
            buckets["1_to_4_prior"].append(ev["anomaly_score"])
        else:
            buckets["5_plus_prior"].append(ev["anomaly_score"])
        seen_count[eid] = n_prior + 1
    return {k: {"n": len(v), "mean_anomaly_score": round(float(np.mean(v)), 4) if v else None}
            for k, v in buckets.items()}


def concept_drift_eval(anomaly_events: List[dict], ground_truth: Dict[str, dict]):
    """Confirms the insider_drift entity is NOT permanently flagged: compares
    its EARLY drift-window scores to its LATE drift-window scores, in the
    order they were scored (scoring must have been run in timestamp order)."""
    drift_events = [ev for ev in anomaly_events
                    if ground_truth[ev["session_id"]]["anomaly_type"] == "insider_drift"]
    if not drift_events:
        return {"note": "no insider_drift sessions found"}
    half = len(drift_events) // 2
    early = [e["anomaly_score"] for e in drift_events[:half]] or [0]
    late = [e["anomaly_score"] for e in drift_events[half:]] or [0]
    return {
        "n_sessions": len(drift_events),
        "early_scores": [round(s, 4) for s in early],
        "late_scores": [round(s, 4) for s in late],
        "mean_early": round(float(np.mean(early)), 4),
        "mean_late": round(float(np.mean(late)), 4),
        "adapted": float(np.mean(late)) < float(np.mean(early)),
    }
def true_positive_classified_events(classified_events: List[dict], ground_truth: Dict[str, dict]):
    """Isolates 'given we correctly flagged it, did we classify it right' —
    the real Layer 2 evaluation criterion — from Layer 1's false positives,
    which have no correct class to measure against."""
    return [c for c in classified_events if ground_truth[c["session_id"]]["label"] == "attack"]


def false_positive_misrouting(classified_events: List[dict], ground_truth: Dict[str, dict]):
    """Diagnostic only: where did Layer 1's false positives get routed by
    the classifier? Kept separate so it doesn't contaminate the
    classification-accuracy number. Useful content for the report's
    limitations section."""
    fps = [c for c in classified_events if ground_truth[c["session_id"]]["label"] != "attack"]
    counts: Dict[str, int] = {}
    for c in fps:
        counts[c["anomaly_type"]] = counts.get(c["anomaly_type"], 0) + 1
    return {"n_false_positives_classified": len(fps), "misrouted_to": counts}
```